Regular Languages

A brief introduction to alphabets, strings and languages

Vivek Kalita

Department of Physics, IISER Mohali

20 October 2026

Contents

Some basic definitions **1**

1.1 Languages and basic operations 2

Languages and the problem of finite representation **4**

2.1 Representation of a language 4

2.2 Can you represent any langauge? 5

Regular Languages **5**

3.1 Regular Languages 6

1 Some basic definitions

Here I introduce some very basic words.

Definition 1.1 (Alphabet, string).

The following are two fundamental building blocks of formal language theory:

1. Alphabet: An alphabet Σ is a finite, non-empty set of symbols, whose elements are called letters.
2. String: A string over an alphabet Σ is a finite ordered sequence of elements of Σ.
   (a) The length of a string w, denoted by $|w|$, is the number of symbols in the sequence.
   (b) The unique string of length zero is called the empty string and is denoted by ε.

Here are some basic examples:

1. Binary alphabet. Consider the alphabet
$$\Sigma = \{0, 1\}.$$
Some strings over this alphabet are
$$0, \quad 1, \quad 001, \quad 1010, \quad 11001.$$
2. English alphabet. Consider the alphabet
$$\Sigma = \{a, b, c, \ldots, z\}.$$
Some strings over this alphabet are
$$\texttt{automata}, \quad \texttt{theory}, \quad \texttt{computer}.$$

3. Decimal alphabet. Consider the alphabet

$$\Sigma = \{0, 1, 2, \ldots, 9\}.$$

Some strings over this alphabet are

$$123, \quad 2026, \quad 314159.$$

Definition 1.2 (Concatenation).

The concatenation of u and v, denoted by uv, is the string obtained by appending the symbols of v to the end of u. In particular, concatenation satisfies

$$|uv| = |u| + |v|,$$

and the empty string ε acts as the identity:

$$u\varepsilon = \varepsilon u = u.$$

The notation Σ^* denotes the set of all finite strings over an alphabet Σ, including the empty string ε.

Let $u, v, w \in \Sigma^*$. The concatenation operation satisfies the following properties:

1. Closure. The concatenation of two strings is again a string over Σ:

$$uv \in \Sigma^*.$$

2. Associativity. Concatenation is associative:

$$(uv)w = u(vw).$$

3. Identity element. The empty string ε acts as the identity element:

$$u\varepsilon = \varepsilon u = u.$$

4. Non-commutativity. Concatenation is not commutative in general:

$$uv \neq vu.$$

It is also important to observe that non-empty strings do not possess inverses under concatenation. This follows immediately from the fact that

$$|uv| = |u| + |v|,$$

which implies that $uv \neq \varepsilon$ whenever $u \neq \varepsilon$. Consequently, the algebraic structure $(\Sigma^*, \cdot)$, where $\cdot$ denotes concatenation, forms a monoid, but not a group.

1.1 Languages and basic operations

Definition 1.3 (Language).

Let Σ be an alphabet. A language L over Σ is any subset of Σ^*, where Σ^* denotes the set of all finite strings over Σ, including the empty string ε. Thus,

$$L \subseteq \Sigma^*.$$

Some examples of languages are given below:

1. Binary language. Consider the alphabet

$$\Sigma = \{0, 1\}.$$

The collection of all binary strings ending in 1 forms a language:

$$L = \{w \in \Sigma^* : w \text{ ends in } 1\}.$$

2. English language. The English language can be regarded as a language over an alphabet consisting of the English letters, spaces, and punctuation marks such as full stops, commas, semicolons, and colons. The collection of valid English expressions forms a language over this alphabet.

3. Empty language. The empty set $\varnothing$ is also a language over any alphabet, since
$$\varnothing \subseteq \Sigma^*.$$

Definition 1.4 (Concatenation of Languages).
Let Σ be an alphabet and let $L_1, L_2 \subseteq \Sigma^*$ be two languages. The concatenation of L_1 and L_2, denoted by L_1L_2, is defined as
$$L_1L_2 = \{uv : u \in L_1,\ v \in L_2\}.$$
In other words, it is the set of all strings obtained by concatenating a string from L_1 with a string from L_2.

Some examples of language concatenation are given below:

1. Let
$$L_1 = \{0, 01\}, \qquad L_2 = \{1, 10\}.$$
Their concatenation is
$$L_1L_2 = \{01, 010, 011, 0110\}.$$
2. Let
$$L_1 = \{a, ab\}, \qquad L_2 = \{b, ba\}.$$
Then
$$L_1L_2 = \{ab, aba, abb, abba\}.$$

The concatenation of languages satisfies the following basic properties. Let $L_1, L_2, L_3 \subseteq \Sigma^*$ be languages.

1. Distributivity over union. Concatenation distributes over union:
$$L_1(L_2 \cup L_3) = L_1L_2 \cup L_1L_3,$$
$$(L_1 \cup L_2)L_3 = L_1L_3 \cup L_2L_3.$$
2. Associativity. Concatenation is associative:
$$(L_1L_2)L_3 = L_1(L_2L_3).$$
3. Identity element. The singleton language $\{\varepsilon\}$ acts as the identity element:
$$L_1\{\varepsilon\} = \{\varepsilon\}L_1 = L_1.$$

Definition 1.5 (Kleene Star).
Let Σ be an alphabet and let $L \subseteq \Sigma^*$ be a language. The Kleene star of L, denoted by L^*, is the set of all strings obtained by concatenating zero or more strings from L. Formally,
$$L^* = \bigcup_{n=0}^{\infty} L^n,$$
where
$$L^0 = \{\varepsilon\}, \qquad L^{n+1} = L^nL.$$

Some examples of the Kleene star operator are given below:

1. Consider the language
$$L = \{a\}.$$
Its Kleene star is
$$L^* = \{\varepsilon, a, aa, aaa, \ldots\}.$$
2. Consider the language
$$L = \{0, 1\}.$$
Then
$$L^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, \ldots\},$$
which is precisely the set of all finite binary strings.

The Kleene star operator satisfies the following basic properties:

1. Empty string. The empty string always belongs to the Kleene star of a language:
$$\varepsilon \in L^*.$$
2. Containment. Every string in L also belongs to L^*:
$$L \subseteq L^*.$$
3. Closure under concatenation. The Kleene star of a language is closed under concatenation. That is, if $x, y \in L^*$, then
$$xy \in L^*.$$
4. Interaction with union. For any two languages L_1, L_2,
$$(L_1 \cup L_2)^* = (L_1^* L_2^*)^*.$$

Remark 1.6

The Kleene star of the empty language is
$$\varnothing^* = \{\varepsilon\}.$$
Indeed, by definition, $L^0 = \{\varepsilon\}$, whereas $L^n = \varnothing$ for every $n \geq 1$ when $L = \varnothing$. Thus, the only string in $\varnothing^*$ is the empty string.

2 Languages and the problem of finite representation

Theorem 2.1

For any finite, non-empty alphabet Σ, the collection of all languages over Σ, namely $\mathcal{P}(\Sigma^*)$, is uncountably infinite.

Proof. Since Σ is finite and non-empty, Σ^* is countably infinite. The result directly follows from Cantor's theorem. □

2.1 Representation of a language

Consider the binary alphabet $\Sigma = \{0, 1\}$. Over this, we are going to desribe a few ways to write languages.

1. Let L_1 be the language consisting of all binary strings ending in 1. That is,
$$L_1 = \{w \in \Sigma^* : w \text{ ends in } 1\}.$$
One representation of this language is the expression
$$(0 \cup 1)^* \cdot 1.$$
Here, $0 \cup 1$ represents the language $\{0, 1\}$, while $\cdot$ denotes concatenation and * denotes the Kleene star.
2. Consider the language
$$L_2 = \{1^n 0^n : n \geq 0\} = \{\varepsilon, 10, 1100, 111000, 11110000, \ldots\}$$
A representation of this language is the recursion:
$$S \longrightarrow 1S0 \mid \varepsilon.$$
Here, S is the starting symbol, and the production rule specifies that S can be replaced either by $1S0$ or by ε.

Notice that the symbols used to construct these representations form two different finite alphabets. In the first case, we have

$$\Gamma_1 = \{0, 1, \cup, \cdot, *, (,)\},$$

whereas in the second case, we have

$$\Gamma_2 = \{S, \longrightarrow, |, \varepsilon, 1, 0\}.$$

We can now formally define what it means to represent a language.

Definition 2.2 (Representation of a Language).

Let Σ be a finite alphabet and let Γ be another finite alphabet used to construct descriptions. Let $D \subseteq \Gamma^*$ be the collection of all valid descriptions.
A representation scheme for languages over Σ is a function

$$\Phi : D \longrightarrow \mathcal{P}(\Sigma^*),$$

which assigns to each valid description $d \in D$ a language $\Phi(d)$ over Σ.
In particular, a description $d \in D$ is said to represent a language L if

$$\Phi(d) = L.$$

2.2 Can you represent any langauge?

Let Γ be a finite, non-empty alphabet used to construct descriptions. Then

$$\Gamma^* = \{\text{all finite strings over } \Gamma\}.$$

Let $D \subseteq \Gamma^*$ denote the collection of all valid descriptions.

Since Γ is finite and non-empty, Γ^* is **countably infinite**. Consequently, D is at most **countably infinite**.

Now, since Σ is also a finite, non-empty alphabet, Σ^* is countably infinite. However, the power set $\mathcal{P}(\Sigma^*)$, which contains all possible languages over Σ, is **uncountably infinite** by Cantor's theorem.

We now consider a representation function

$$\Phi : D \longrightarrow \mathcal{P}(\Sigma^*).$$

For this representation scheme to describe every possible language over Σ, the function Φ must be surjective.

However, since $|D| < \mathcal{P}(\Sigma^*)$, such a function cannot be surjective.

Theorem 2.3

There exists no finite-description alphabet Γ that can represent every possible language over a given finite, non-empty alphabet Σ.

3 Regular Languages

Definition 3.1 (Regular Expressions).

Fix an alphabet Σ. The regular expressions over Σ are defined recursively by the following rules:

- R1. The symbol $\varnothing$ and every symbol $a \in \Sigma$ are regular expressions.
- R2. If α and β are regular expressions, then their concatenation $(\alpha\beta)$ is a regular expression.
- R3. If α and β are regular expressions, then their union $(\alpha \cup \beta)$ is a regular expression.
- R4. If α is a regular expression, then α^* is a regular expression.
- R5. Nothing is a regular expression unless it can be obtained by finitely many applications of the preceding rules.

Consider the alphabet

$$\Sigma = \{0, 1\}.$$

We want to represent the language consisting of all binary strings containing the substring 101.

One regular expression representing this language is

$$(0 \cup 1)^* 101 (0 \cup 1)^*.$$

To understand this expression, consider the following:

1. The expression
$$(0 \cup 1)^* = \{0, 1\}^*$$
represents the set of all finite binary strings, including the empty string.

2. The expression
$$(0 \cup 1)^* 101 (0 \cup 1)^* = \{0, 1\}^* \{101\} \{0, 1\}^* = \{101,\ 0101,\ 1010,\ 11010,\ 0010111,\ \ldots\}$$
represents all binary strings containing the substring 101.

3.1 Regular Languages

Definition 3.2 (Regular Language).
A language $L \subseteq \Sigma^*$ is said to be regular if there exists a regular expression α over Σ such that

$$L = \mathcal{L}(\alpha),$$

where $\mathcal{L}(\alpha)$ denotes the language generated by the regular expression α.

Definition 3.3 (Recursive Definition of Regular Language).
Let α and β be regular expressions over an alphabet Σ. The language generated by a regular expression is defined recursively as follows:

1. Base cases.
   - B1. $\mathcal{L}(\varnothing) = \varnothing$.
   - B2. $\mathcal{L}(\varepsilon) = \{\varepsilon\}$.
   - B3. For every $a \in \Sigma$, $\mathcal{L}(a) = \{a\}$.
2. Recursive cases.
   - R1. $\mathcal{L}(\alpha \cup \beta) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$.
   - R2. $\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha)\mathcal{L}(\beta)$.
   - R3. $\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^*$.